

AI-Assisted Development in Practice

Plan, prompt, implement, verify, review - without surrendering ownership.

AI changes the bottleneck from typing to reviewing.

Learning outcomes

What students should be able to do after this module

- Use Frame → Plan → Prompt → Execute → Verify → Reflect
- Ask for implementation plans before code
- Use AI for debugging, refactoring, tests, docs, API clients, and review
- Work in existing codebases with Map → Bound → Verify
- Critique generated tests and generated diffs
- Prevent sensitive-data leakage in prompts

**The goal is not perfect prompts.
The goal is controlled engineering
workflow.**

Practical anchor

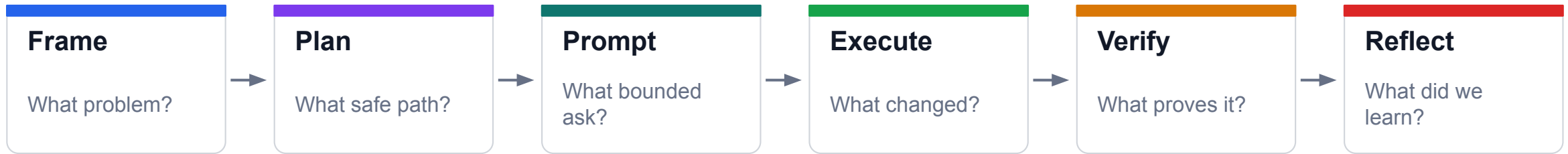
A small FastAPI Work Items API with a failing starter repo and a passing solution repo.

Review question

Did AI make the work more reviewable - or merely faster?

The workflow spine

Frame → Plan → Prompt → Execute → Verify → Reflect



The assistant participates in the loop. It does not replace the loop.

For nontrivial tasks, ask for a plan before code.

AI shifts the bottleneck

More output means more review work

Before AI assistance

Developer time was often spent typing, searching, and writing repetitive scaffolding.



After AI assistance

Output appears faster: code, tests, docs, PR summaries, client code, and review comments.

- ❑ Generated output can be useful
- ❑ Generated output can be plausible but incomplete
- ❑ Review capacity becomes the limiter
- ❑ Tests and diff review become central

- ❑ Look for scope creep
- ❑ Inspect assertions, not just green tests
- ❑ Challenge assumptions
- ❑ Keep production risk human-owned

Frame the task before the prompt

Prompt quality starts as engineering clarity

- ☐ What behavior should change?
- ☐ What must stay stable?
- ☐ What context is safe to share?
- ☐ What is the smallest useful slice?
- ☐ How will we verify it?

Weak frame

“Improve the API.”

Stronger frame

“Complete missing repository behavior for CRUD work items. Keep endpoint response shapes stable. Start with a plan and tests.”

The frame is where ownership starts.

Plan before code

Use AI to expose assumptions before it edits files

Prompt

Do not write code yet.

Inspect this FastAPI project and propose a minimal implementation plan.

Include files, out-of-scope files, risks, tests, and verification commands.

A good plan reveals

likely files · scope creep · missing tests · unsafe assumptions · verification burden

Instructor move

Ask: “What would you reject or narrow before implementation?”

Expected boundary

For the main lab, the first slice should primarily touch `app/repository.py`.

Prompting for developers: five layers

Task + Context + Constraints + Output + Verification

Task

What should change?

Context

Files, tests, ticket, stack trace

Constraints

What must stay stable?

Output

Plan, diff, tests, checklist, docs

Verification

Commands, manual checks, risks

Prompt quality is mostly engineering clarity.

- ☐ Tell the assistant what not to do
- ☐ Require assumptions and risks
- ☐ Use output formats that are easy to review
- ☐ Always ask for verification

Prompt rescue exercise

Add boundaries to weak prompts

Weak

Fix this test.



Better

Explain why this test fails. List likely causes and the smallest check before changing code.

Weak

Write tests.



Better

Write pytest tests for behavior, validation failures, and edge cases. Keep assertions explicit.

Weak

Review my code.



Better

Review this diff as a review aid. Identify behavior changes, missing tests, privacy/security concerns, and reviewer questions.

Repository context is infrastructure

Instructions, tests, docs, and conventions shape AI output

README

How to run and understand the project

Tests

Executable expectations

AI instructions

Persistent constraints and workflow rules

Prompt files

Reusable task patterns

Review checklist

Human review criteria

A repo that is legible to humans is usually more legible to AI.

Daily workflow beyond code generation

The assistant can help across the workday

Ticket

clarify scope
acceptance criteria

Code

bounded edit
refactor

Debug

hypotheses
evidence

Tests

draft + critique
edge cases

Docs

actual behavior
examples

PR / release

summary
review checklist

Repeated pattern: ask → inspect → verify → decide.

Guided demo: build a REST API slice

AI assistance only — but human discipline throughout



1. Run failing tests
2. Ask for plan, not code
3. Implement create/list first
4. Run targeted tests
5. Implement remaining CRUD + filters
6. Review diff and PR summary

**Demo success is not “AI wrote code.”
Demo success is when the decision points
are understood.**

Lab 1: Work Items API

Hands-on implementation, tests, verification, prompt log

1. Start with the failing starter repo
2. Ask for a plan before writing code
3. Complete create/list/get/update/delete behavior
4. Support status and priority filters
5. Review generated tests
6. Record one AI suggestion you rejected or revised

Definition of done

working API · passing tests · reviewed diff · prompt log · rejected/revised suggestion

Do not do this

Do not delete tests, rewrite the app broadly, or change response shapes just to satisfy generated code.

Optional challenge

Pagination, due date, tags, sorting, or CI badge.

Debugging with AI

Ask for hypotheses before fixes

Better debugging prompt

Do not patch yet.

Analyze this failing test and stack trace.

List the top 3 likely causes, evidence for and against each, the smallest check to confirm, and the first fix you would try.

- ☐ Prevents random patching
- ☐ Separates diagnosis from remediation
- ☐ Creates better debriefs
- ☐ Keeps evidence visible

Debugging is a reasoning loop, not a code-generation task.

Generated tests can be wrong too

A generated test file is a draft, not proof

Looks useful

Several tests, nice names, quick to produce

Often weak

Only status codes, no mixed data, no payload assertions, misleading names

Your job

Review assertions, edge cases, and behavior proven

- ☐ A test name can lie
- ☐ Coverage is not correctness
- ☐ Happy-path tests are not enough
- ☐ Ask AI to critique the tests it generated

The assertion is where truth lives.

Refactoring and comments

Tests before refactor; comments explain intent

Safe refactor loop

Explain behavior → add tests → plan refactor → small edit → run tests → review diff

- ☐ Do not refactor unprotected behavior
- ☐ Ask for behavior-preserving plans
- ☐ Remove comments that merely restate code
- ☐ Document caveats and actual behavior

Weak comment

```
# send request  
response = client.get(...)
```

Better docstring

```
“List work items using server-side filters so callers avoid  
fetching unnecessary data.”
```

API client generation and review

Required compact coverage for API clients

Prompt for client

Generate a small Python httpx client with explicit timeouts, clear errors, docstrings, and no invented auth.

Review checklist

timeouts · errors · response assumptions · useful comments · no fake secrets · MockTransport tests

- ☐ AI can scaffold the client quickly
- ☐ Human review checks operational assumptions
- ☐ Generated comments often need pruning
- ☐ Advanced retries/auth are optional extensions

**Basic API-client coverage is required.
Advanced hardening is optional.**

Legacy code explanation

AI is useful when the code already exists

Developer

functions · data flow · edge cases · tests

Product

behavior · user impact · limitations

Support

symptoms · troubleshooting · known caveats

Crucial follow-up

Which claims are directly visible in the code, and which are assumptions?

Lab 2: legacy tests and docs

Understand first. Test second. Change only when protected.

- ☐ Ask for a read-only explanation
- ☐ Mark vague or suspicious claims
- ☐ Generate edge cases
- ☐ Draft tests and review assertions
- ☐ Write documentation matching actual behavior
- ☐ Optional: propose a refactor plan without implementing

Source

lab2_assets/legacy_work_item_logic.py

Expected AI overclaim

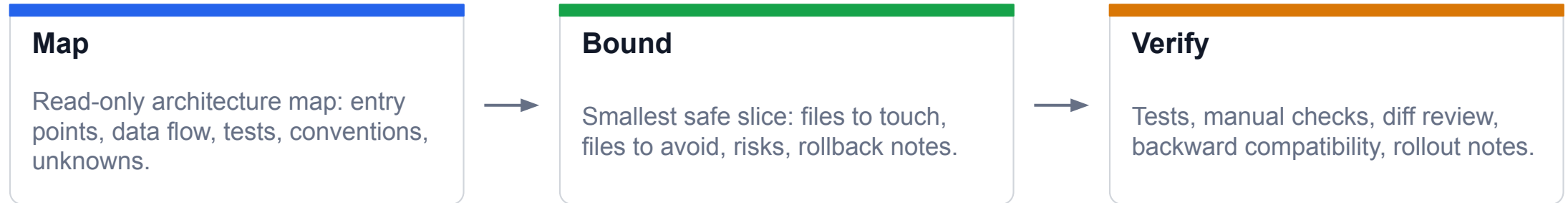
May overstate robustness of date parsing.

Optional part

Behavior-preserving refactor plan after tests exist.

Existing codebases: Map → Bound → Verify

A safe adoption ladder for real repos



Explain → Test → Tiny fix.

Review discipline for AI-generated diffs

AI can assist review, but cannot approve

AI can help

summaries · risk areas · missing tests · reviewer questions · PR draft

AI cannot own

approval · accountability · security sign-off · architecture fit · merge decision

- ☐ Review behavior changes
- ☐ Check for scope creep
- ☐ Inspect generated tests
- ☐ Look for privacy/security issues
- ☐ Ask what manual checks remain

Use AI as a review amplifier, not a rubber stamp.

Context awareness and leakage prevention

Before pasting, classify and redact

- ☐ Secrets, tokens, credentials
- ☐ Customer data and personal data
- ☐ Private logs and stack traces
- ☐ Internal URLs and architecture details
- ☐ Unreleased product plans or vulnerability details

Safer prompt pattern

“Here is a sanitized example with fake IDs and no secrets. Diagnose the pattern and suggest checks I can run locally.”

Do not paste what you would not put in a broad ticket comment.

Optional accelerators once basics work

OPTIONAL

Custom instructions

Persistent repo rules

Prompt files

Reusable task patterns

Code review aids

Checklist + summaries

MCP preview

Tool-connected context; higher risk

Cloud agents

Issue-shaped task; PR review

Do not skip the basics: plan, bound, verify, review.

Optional: terminal and cloud-agent comparison

OPTIONAL

Higher autonomy means higher review burden

Terminal / CLI agent

Useful for command-oriented repo work. Requires command explanation, approval, and rollback awareness.

Cloud / coding agent

Useful for issue-shaped tasks. Requires acceptance criteria, out-of-scope boundaries, tests, and PR review.

Do not delegate

deployments · secret rotation · production data mutation · incident response decisions

Optional homework and extensions

OPTIONAL

Every exercise has a suggested solution

API extension

due date · pagination · tags · soft delete
· sorting

Legacy onboarding

behavior · assumptions · risks · missing
tests

API client review

timeouts · errors · useful comments · no
fake auth

Prompt cookbook

planning · debugging · review prompts

Reflection

AI suggestions accepted, rejected,
revised; verification evidence

Takeaways

- ❑ Ask for a plan before code when the task is nontrivial
- ❑ Keep changes bounded and reviewable
- ❑ Generated tests are drafts, not proof
- ❑ Existing repos need Map → Bound → Verify
- ❑ API clients, comments, and docs require assumption review
- ❑ AI review is a review aid, not approval
- ❑ Developers still own correctness, security, and maintainability

Working code is not yet owned code!

Module 3 turns up generation speed - and the ownership problem.

Exit question: When should you ask AI for a plan instead of code?